

Functions

Introduction to Programming

StayAhead Training; Sep, 2024

Introduction

Functions

- **Function:** a name for some code stored in memory
- Function AKA method, procedure, subroutine
- Technically methods etc. differ from functions

Declaration

Functions

- Pseudocode, e.g.:
 - FUNCTION Greet()
 - OUTPUT "Hello!" to stdout
 - OUTPUT "How are you?" to stdout
 - ENDFUNCTION

Declaration

Functions

- `def greet():`
- `print("Hello!")`
- `print("How are you?")`

Invocation/Call Functions

- Pseudocode, e.g.:
 - CALL Greet()

Invocation/Call Functions

- `greet()`

Invocation (order of execution)

Functions

- `def greet():`
- 3. `print("Hello!")`
- 4. `print("How are you?")`
- 1. `print("Welcome to my program")`
- 2. `greet()`
- 5. `print("Goodbye")`

Parameters

Functions

- **Parameter:** a name for some data passed to a function
- A parameter is a variable
- A parameter is assigned a value when the function is called

Parameters

Functions

- Pseudocode, e.g.:
 - FUNCTION Greet(Name)
 - OUTPUT "Hello" + Name to stdout
 - OUTPUT "How are you?" to stdout
 - ENDFUNCTION

Parameters

Functions

- `def greet(name):`
- `print("Hello " + name)`
- `print("How are you?")`

Arguments

Functions

- **Argument:** some data passed to a function at call time
- The argument is the value assigned to the parameter

Arguments (literals)

Functions

- `greet("Harold")`

Arguments (variables)

Functions

- `greet(harry)`

Arguments

Functions

- When declaring a function you choose the parameter names
- When calling a function the variables you pass to it as arguments need not have the same names as the corresponding parameters

Parameters (more than one)

Functions

- `def greet(name, age):`
- `yearOfBirth = 2024 - age`
- `print("Hello " + name)`
- `print("You were born in " + str(yearOfBirth))`

Arguments (more than one)

Functions

- `greet("Harold", 75)`
- `greet(name, age)`

Parameters (complex types)

Functions

- `def print_all(array):`
- `for element in array:`
- `print(element)`

Return Value

Functions

- **Return value:** a value returned/output by a function
- A function can return only one thing
- The one thing returned may be complex, e.g. an array

Return Value

Functions

- Pseudocode, e.g.:
- FUNCTION GetSum(Numbers)
- SET Sum to 0
- FOREACH Number in Numbers
- COMPUTE Sum = Sum + Number
- ENDFOREACH
- RETURN Sum
- ENDFUNCTION

Return Value

Functions

- `def get_sum(numbers):`
- `sum = 0`
- `for number in numbers:`
- `sum += number`
- `return sum`

Return Value

Functions

- Pseudocode, e.g.:
 - SET Nums to [1, 2, 3, 4, 5]
 - COMPUTE Total = CALL GetSum(Nums)

Return Value

Functions

- `sum = get_sum(numbers)`

Output as Input

Functions

- `print(get_sum(numbers))`

Variable Scope

Functions

- **Variable scope:** the durability and accessibility of a variable
- For how long does the variable exist in memory?
- To which other parts of the code (other functions) is the variable accessible?

Variable Scope

Functions

- **Global variable:** one that is declared outside of a function
- A global variables exists in memory for as long as the program runs
- A global variable is accessible to all program/module functions

Global Variables

Functions

- `global = 1`
- `print(global)`
- `def print_global():`
- `print(global)`

Variable Scope

Functions

- **Local variable:** one that is declared inside a function
- A local variable exists in memory only while the function is executing
- A local variable is only accessible to its enclosing function

Local Variables

Functions

- `def my_first_function():`
- `first = 1`
- `def my_second_function():`
- `print(first) # runtime error`
- `print(first) # runtime error`

Variable Scope

Functions

- Each global variable name must be unique
- Two or more local variables in the same function must have unique names
- A local variable may share its name with a local variable in another function

Modules and Libraries

Functions

- **Module:** a file containing related code
- Large programs are often comprised of many modules
- **Library:** a collection of modules

Procedural Programming

Functions

- A top-down approach to program design & development
- Break the problem down into smaller and smaller ones
- Keep going until the problem is small enough to solve with a function
- This process is known as functional decomposition